Assignment 1 Report

Raquel Ana Magalhães Bush

Professor Asif Turzo

CIS 546 Secure Software Development

5 March 2026

Startup Process

Because I am on a Mac, I had to replicate what startup.bat does on Windows.

startup.bat calls webserv.bat, which does php src/Startup.php to populates the initial database and php -S localhost:8000 to open the web app in the browser.

So, I did these commands myself in the terminal to start the web application:

```
(base) raquelanabush@Raquels-MacBook-Pro-3 app % php src/Startup.php
(base) raquelanabush@Raquels-MacBook-Pro-3 app % php -S localhost:8000
[Sat Feb 28 12:40:40 2026] PHP 8.5.3 Development Server (http://localhost:8000) started
[Sat Feb 28 12:40:50 2026] [::1]:61191 Accepted
[Sat Feb 28 12:40:50 2026] [::1]:61191 [200]: GET /public/index.php
[Sat Feb 28 12:40:50 2026] [::1]:61191 Closing
[Sat Feb 28 12:40:50 2026] [::1]:61192 Accepted
[Sat Feb 28 12:40:50 2026] [::1]:61192 [200]: GET /resources/secure_app.css
[Sat Feb 28 12:40:50 2026] [::1]:61193 Accepted
[Sat Feb 28 12:40:50 2026] [::1]:61192 Closing
[Sat Feb 28 12:40:50 2026] [::1]:61193 [200]: GET /resources/Header_Lock_Image.svg
[Sat Feb 28 12:40:50 2026] [::1]:61193 Closing
```

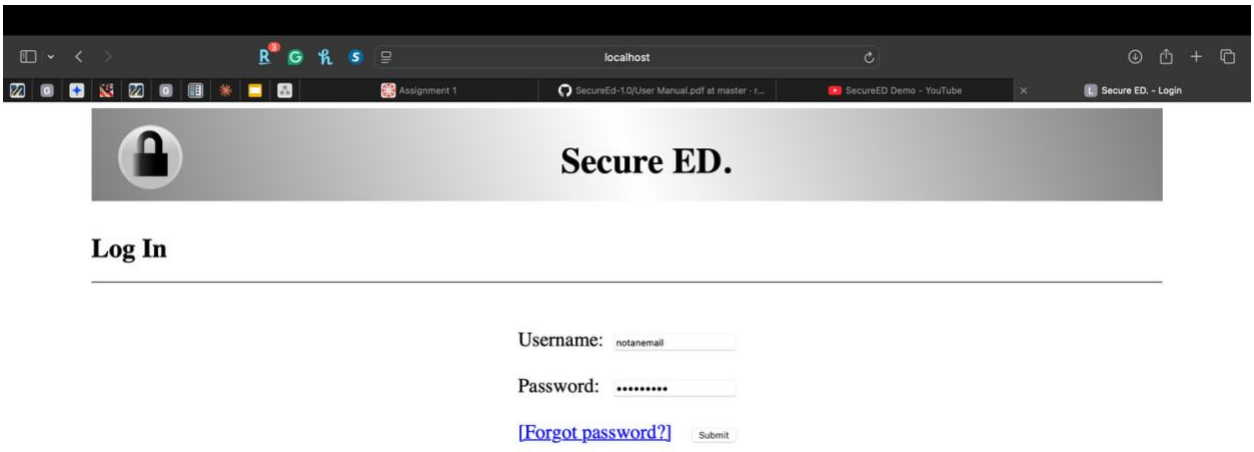
Task 1: Applying input validation

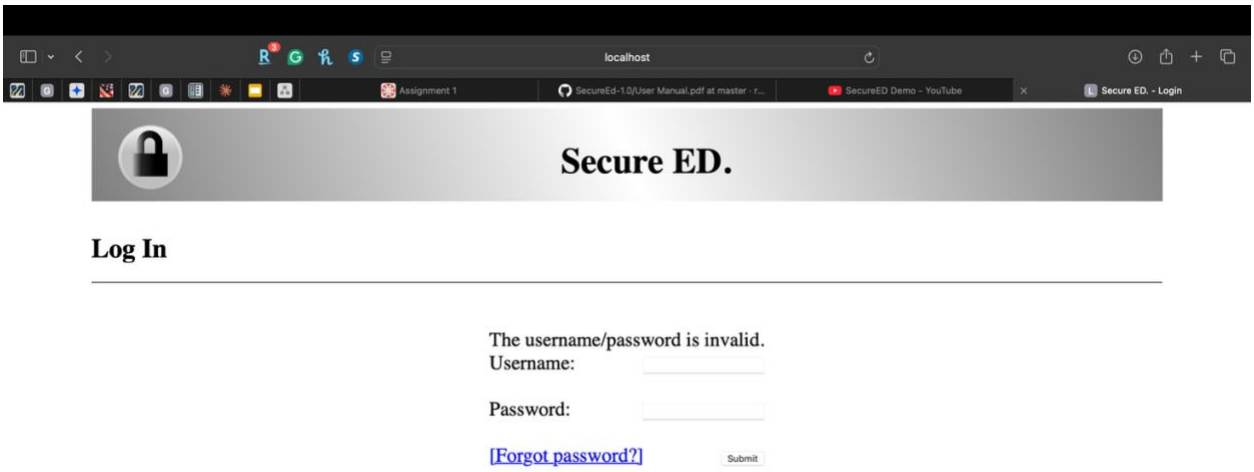
I applied input validation to login.php using regular expressions to ensure that the username is a valid email address and that the password is a valid text with length between 8 to 16 (inclusive) consisting of characters a-z, A-Z, and 0-9 with at least one capital letter and one number:

```php
# TASK 1: Applying input validation
/*Get information from the post request*/
$myusername = $_POST['username'] ?? ''; # default to empty string if not provided
$mypassword = $_POST['password'] ?? ''; # default to empty string if not provided

# moved up for logical flow, normalization should happen before validation:
$myusername = strtolower($myusername); //makes username noncase-sensitive

// email validation:
if (!preg_match("/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/", $myusername)) {
    header("Location: ../public/index.php?login=fail");
    exit();
}

// password validation:
if (!preg_match("/^(?=.*[A-Z])(?=.*[0-9])[A-Za-z0-9]{8,16}$/", $mypassword)) {
    header("Location: ../public/index.php?login=fail");
    exit();
}

//convert password to 80 byte hash using ripemd256 before comparing
$hashpassword = hash('ripemd256', $mypassword);

#if($myusername==null)
#{throw new Exception("input did not exist");}  # no longer needed due to null coalescing above
```

I used the null coalescing operator (??) to ensure that missing POST inputs default to empty strings, preventing undefined index warnings and improving robustness. I moved username normalization (strtolower) before validation to ensure consistent case-insensitive handling during both validation and authentication. Finally, I removed the unnecessary exception block since inputs are now guaranteed to be defined, reducing the risk of exposing sensitive debugging information.

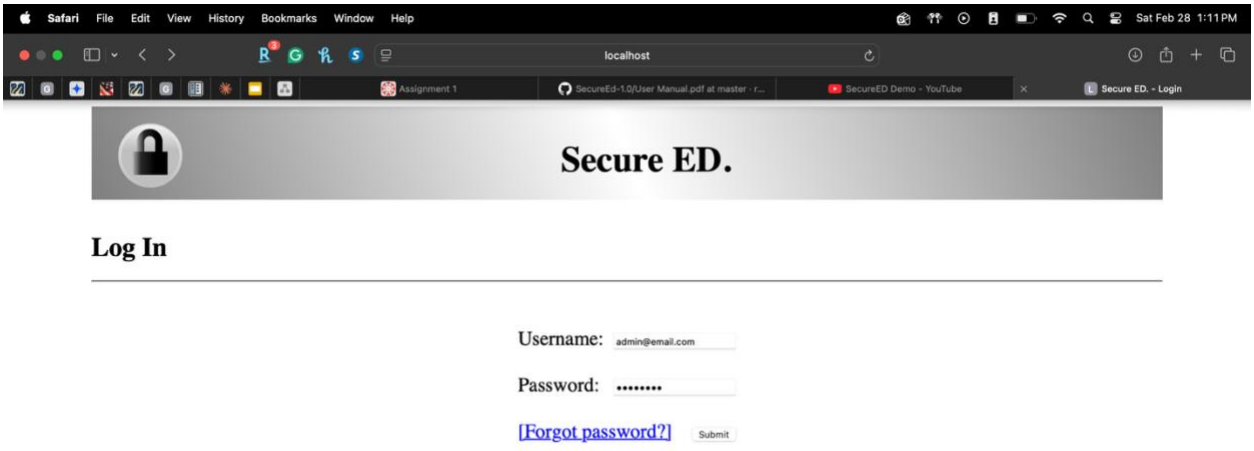
When the username is invalid, we get the general message:

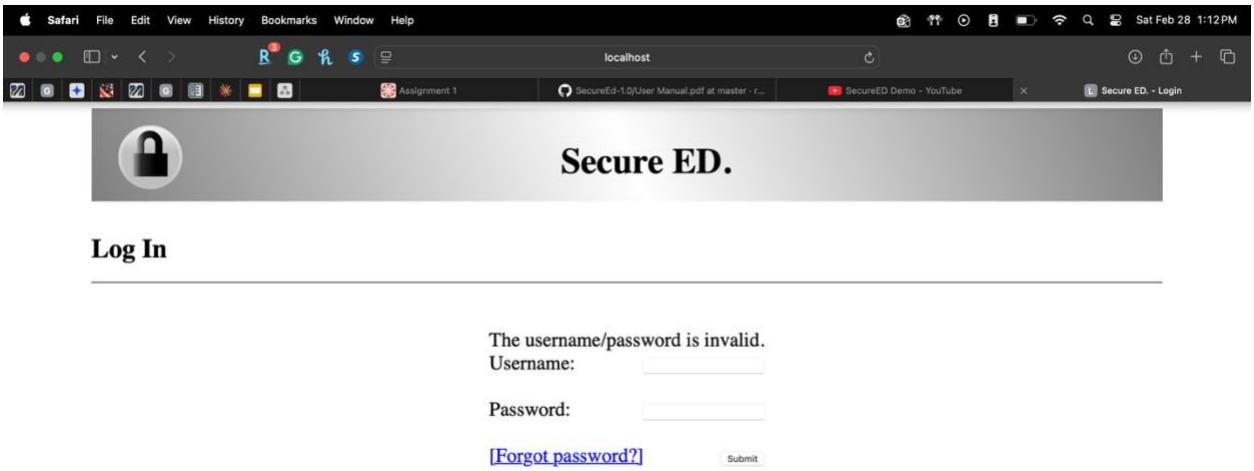




When the password is invalid, we also get the general error message:

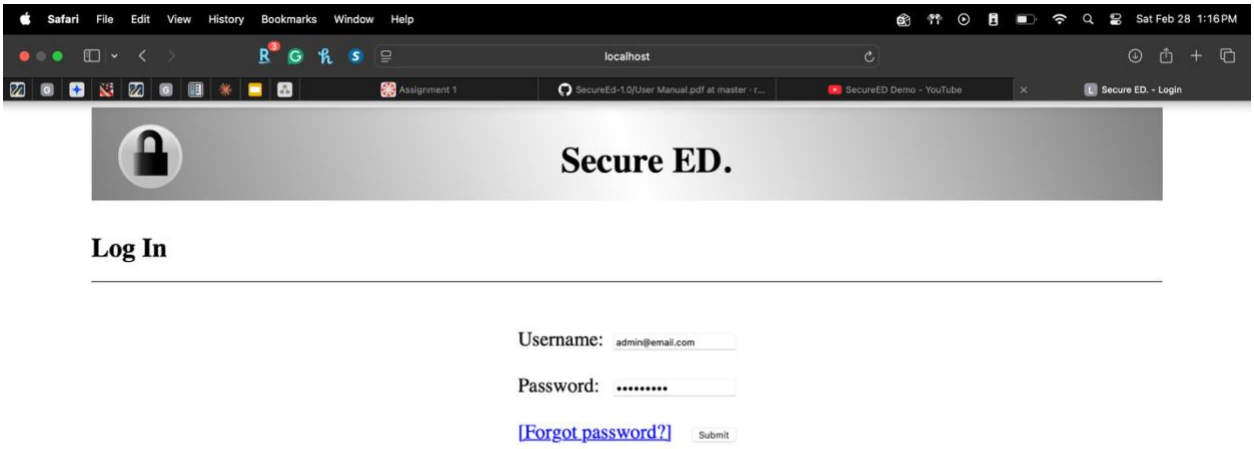
(I entered “password” into the Password field)

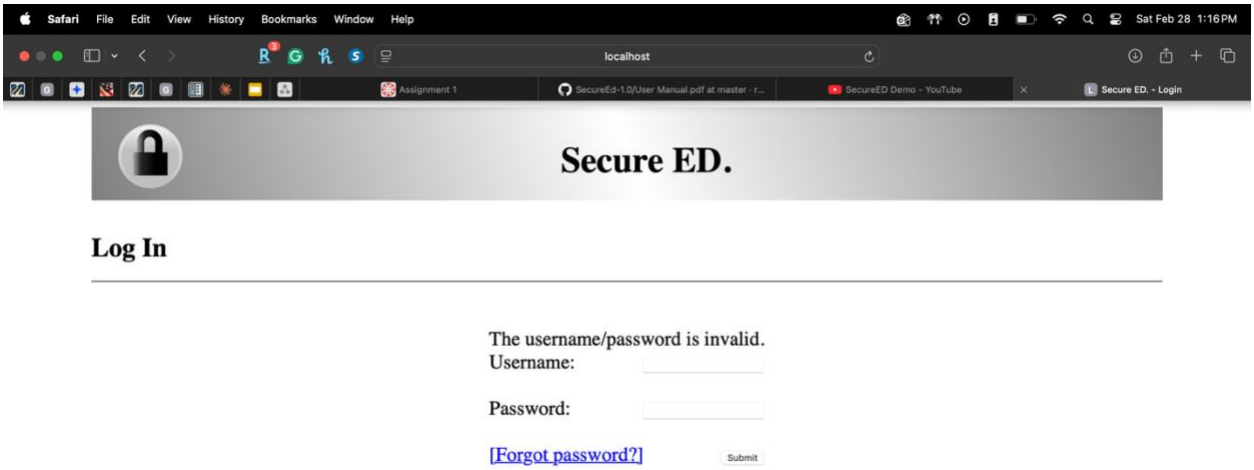




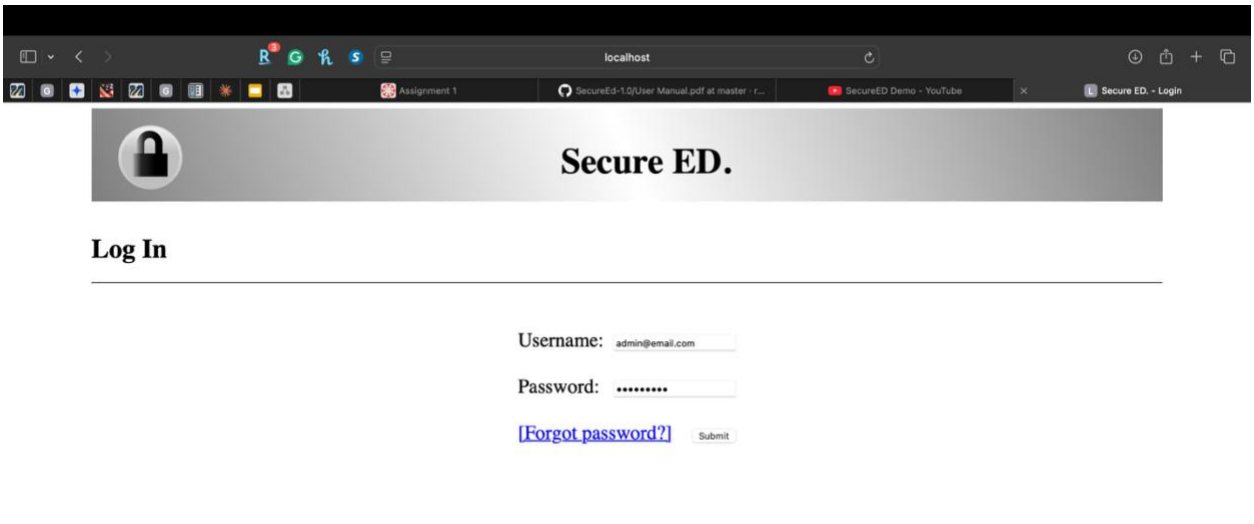
When both are the valid format but the password is incorrect, we also get the general error message:

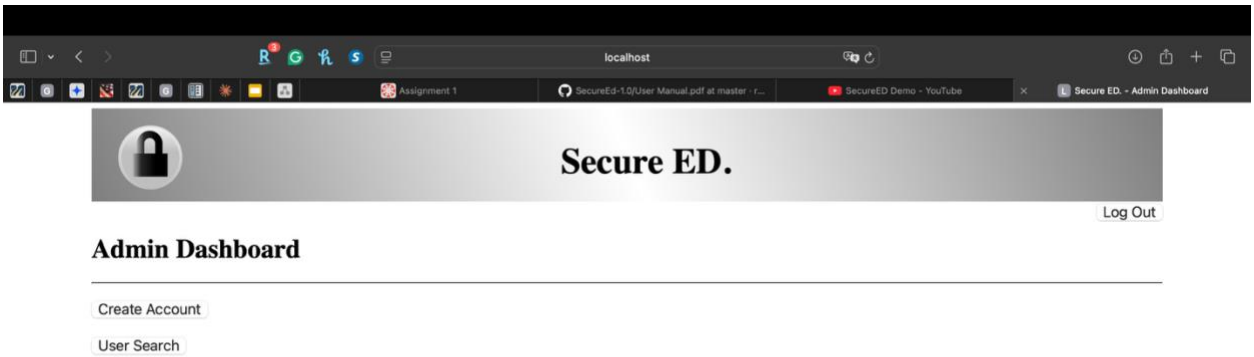
(I entered “Password9” into the Password field, when the correct entry would be “Password1”)





Logging into the web application using the Admin credentials provided in the User Manual results in entry to the SecureED Admin Dashboard:





Task 2: Mitigating the SQLi vulnerability

SQL injection (SQLi) is a vulnerability that occurs when user-supplied input is embedded directly into a SQL query without proper sanitization, allowing an attacker to alter the structure of the query itself. Rather than being treated as data, the malicious input is interpreted as executable SQL, giving the attacker control over what the database executes.

For example, consider a login query that checks credentials using string interpolation:

```
SELECT * FROM User WHERE Email='$email' AND Password='$password'
```

If an attacker enters ' OR '1'='1 as the username, the query becomes:

```
SELECT * FROM User WHERE Email='' OR '1'='1' AND Password='...'
```

Since '1'='1' is always true, the condition evaluates to true for every row in the database, bypassing authentication entirely and potentially granting unauthorized access.

Change #1:

The regex I added to address Task 1 may incidentally exclude ', --, ; etc. (email/alphanum-only patterns), so the app may not be exploitable right now, but input validation is not a SQLi mitigation. It's the wrong layer. Regex can be wrong, can be bypassed, can change, and doesn't communicate intent. The defense must live at the database layer.

The original queries constructed SQL strings by directly embedding user-supplied values ($myusername, $mypassword, $hashpassword) using PHP string interpolation. This meant that if an attacker supplied a crafted input containing SQL metacharacters, such as a single quote to break out of the string literal, they could alter the structure of the query itself, potentially bypassing authentication or extracting data from the database. To mitigate this, I rewrote the queries using SQLite3 prepared statements with named placeholders (:email, :pass, :hash). With prepared statements, the SQL structure is compiled by the database engine first, and user values are bound afterward via bindValue(). They are treated strictly as data, never as executable SQL. This separation of code and data at the database driver level eliminates the SQL injection vulnerability regardless of what characters the user submits, and is the correct, standard defense rather than relying on input validation to accidentally exclude dangerous characters.

Before:

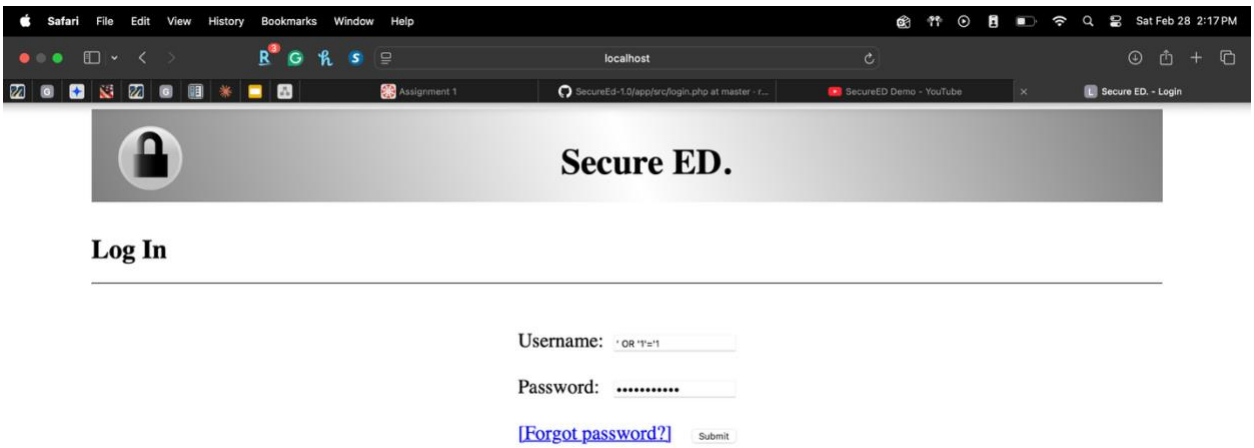
```
//query for count
$query = "SELECT COUNT(*) as count FROM User WHERE Email='$myusername' AND (Password='$mypassword' OR Password='$hashpassword')";
$count = $db->querySingle($query);

//query for the row(s)
$query = "SELECT * FROM User WHERE Email='$myusername' AND (Password='$mypassword' OR Password='$hashpassword')";
$results = $db->query($query);
```

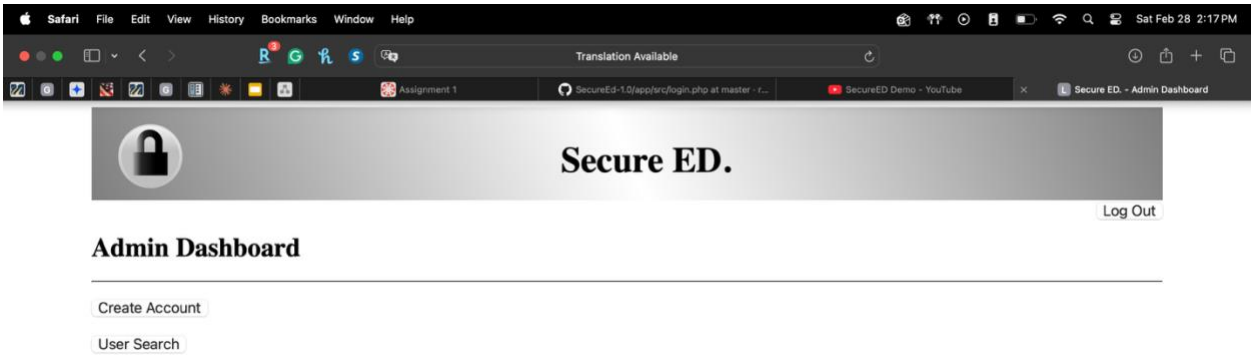
After:

```
# TASK 2: Mitigate the SQLi vulnerability
//query for count
$stmt = $db->prepare("SELECT COUNT(*) as count FROM User WHERE Email=:email AND (Password=:pass OR Password=:hash)");
$stmt->bindValue(':email', $myusername,   SQLITE3_TEXT);
$stmt->bindValue(':pass',  $mypassword,   SQLITE3_TEXT);
$stmt->bindValue(':hash',  $hashpassword, SQLITE3_TEXT);
$countResult = $stmt->execute();
$count = $countResult->fetchArray(SQLITE3_ASSOC)['count'];

//query for the row(s)
$stmt2 = $db->prepare("SELECT * FROM User WHERE Email=:email AND (Password=:pass OR Password=:hash)");
$stmt2->bindValue(':email', $myusername,   SQLITE3_TEXT);
$stmt2->bindValue(':pass',  $mypassword,   SQLITE3_TEXT);
$stmt2->bindValue(':hash',  $hashpassword, SQLITE3_TEXT);
$results = $stmt2->execute();
```

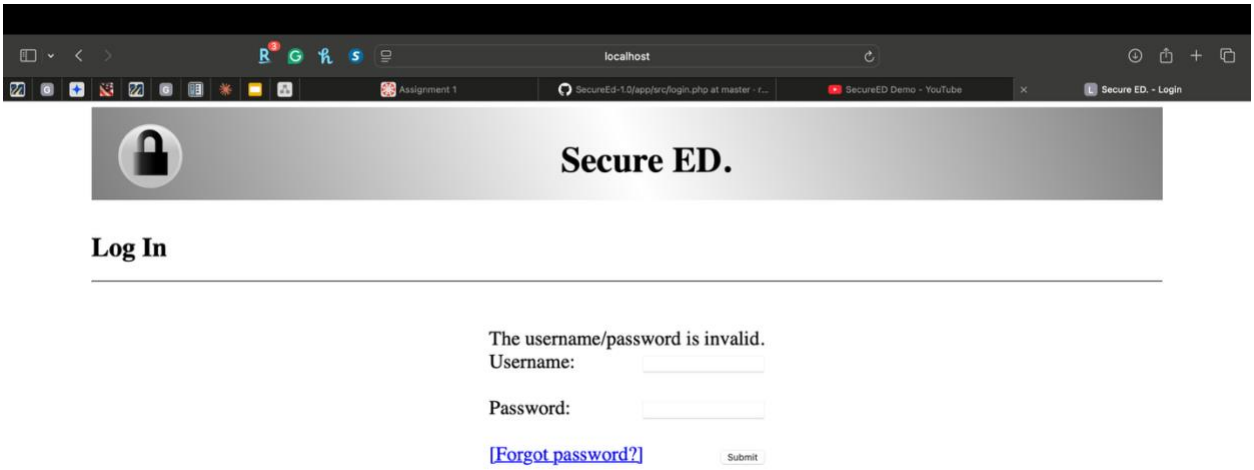
Entering ' OR '1'='1 for both inputs does not break out of the SQL string and change the query's logic, showing that with my prepared statements, that input is treated as a literal string value and the database literally searches for a user whose email is ' OR '1'='1, finds nothing, and returns the same general error message.

Before making these changes, I could break in using SQL injection. Using the payload ' OR '1'='1 in the username field with any password, the original vulnerable code grants access to the Admin Dashboard without valid credentials:





Now, I get the same general error message:



Change #2:

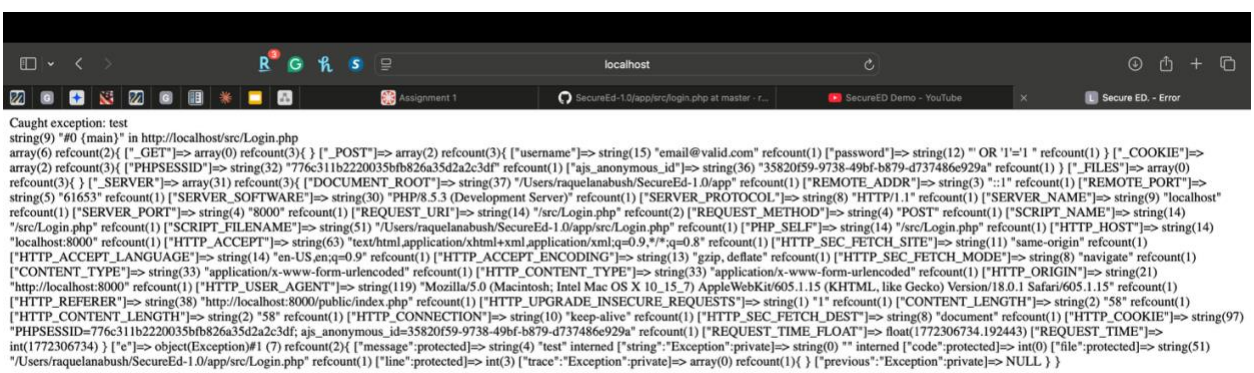
While the prepared statements directly eliminate the SQL injection vulnerability, I also hardened the catch block as a defense-in-depth measure. The original error handler exposed stack traces, variable dumps, query structure, and server details directly in the browser. This is information that an attacker could use to study the application and craft a more targeted injection payload. By redirecting exceptions server-side and showing nothing to the user, this change reduces the reconnaissance value of any errors the application produces, limiting what an attacker can learn if they probe the login endpoint.

To test this, I used the original version of the code without input validation or prepared statements, so that I could trigger the verbose catch block and demonstrate what information it exposed to the browser.

Before:

```php
catch(Exception $e)
{
    //prepare page for content
    include_once "ErrorHeader.php";

    //Display error information
    echo 'Caught exception: ',  $e->getMessage(), "<br>";
    var_dump($e->getTraceAsString());
    echo 'in '.'http://'. $_SERVER['SERVER_NAME'] . $_SERVER['REQUEST_URI']."<br>";

    $allVars = get_defined_vars();
    debug_zval_dump($allVars);
}
```

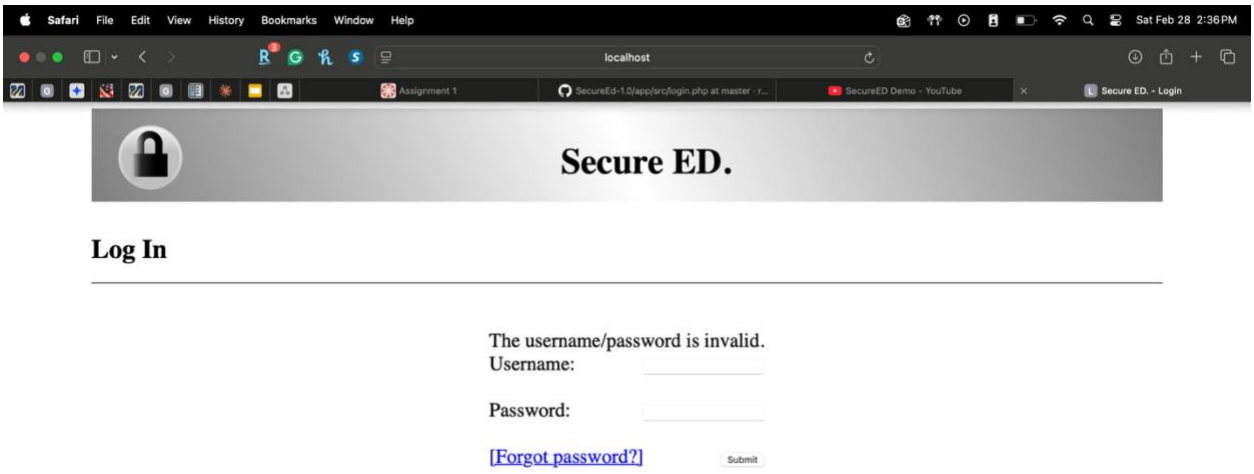
Inserting throw new Exception("test"); in the try block and retrying entry results in the following:



After:

```
catch(Exception $e)
{
    error_log($e->getMessage() . "\n" . $e->getTraceAsString()); // logs server-side only
    header("Location: ../public/index.php?login=fail");
    exit();
}
```

Now when I insert throw new Exception("test"); in the try block and retry entry, it results in the same general error message:



Conclusion

In this assignment, I improved the security of the SecureED login functionality by addressing both input handling and a critical SQL injection vulnerability. I first implemented structured input validation to enforce consistent formatting and improve robustness. While this strengthened input handling, I recognized that validation alone is not a true defense against SQL injection.

To properly eliminate the vulnerability, I changed the login queries to use SQLite prepared statements with bound parameters. This ensures a clear separation between SQL code and user-supplied data, preventing malicious input from altering query structure. I confirmed this mitigation by demonstrating that a previously successful injection payload no longer bypasses authentication.

Finally, I hardened the application’s error handling by removing verbose debugging output from the browser and logging errors server-side. This reduces information disclosure and limits what an attacker can learn from probing the login endpoint.